

K8s Networking & Configuration for AI Workloads

Week 3 · Session 3 · Lab Guide

What will you learn

- How ConfigMaps and Secrets externalise training configuration via envFrom, so the training image never needs to be rebuilt to tune a run or rotate a token.
- Why headless Services (clusterIP: None) plus a StatefulSet are the backbone of distributed training DNS -- and what actually breaks (NCCL rendezvous) if you use a regular ClusterIP Service instead.
- How a LoadBalancer Service exposes a production inference endpoint, versus ClusterIP/NodePort.
- How Gateway API (GatewayClass + Gateway + HTTPRoute) replaces Ingress, decoupling the platform team (owns the Gateway) from app teams (self-serve HTTPRoutes in their own namespace).
- How Istio's DestinationRule + VirtualService implement a canary rollout by traffic weight, and how mTLS between pods is automatic once sidecar injection is enabled.

Before we start

This guide assumes Docker Desktop is installed with the WSL2 backend and Kubernetes enabled (Settings → Kubernetes → Enable Kubernetes). Unlike Session 1's GPU pod demo, none of Session 2 needs a second GPU-aware cluster -- everything here runs on the plain docker-desktop context.

Verify your environment first:

```
kubectl config use-context docker-desktop
kubectl get nodes # should show a Ready control-plane node
```

The Gateway API and Istio demos (Parts 3 and 4 below) need two one-time installs that Session 1 does not: the Gateway API CRDs, and Istio itself. Both are covered in their own sections below with exact commands and the real output captured while testing this guide.

Part 1a — ConfigMaps, Secrets & Config Injection

Folder: 03_networking_config

Files: 04_configmap.yaml, create_secret.sh, 05_training_pod.yaml

What this teaches

A ConfigMap externalises hyperparameters (batch size, learning rate, ...) so the training image never needs rebuilding to tune a run. A Secret holds API tokens (W&B, Hugging Face) the same way, but created imperatively -- never committed to Git as plain YAML with real values. Both are consumed by a pod identically, via envFrom.

How to run it

Step 1 — create the ConfigMap and Secret:

```
cd 03_networking_config
kubectl apply -f 04_configmap.yaml
bash create_secret.sh
```

Step 2 — deploy a pod that consumes both via envFrom, and check the env inside it:

```
kubectl apply -f 05_training_pod.yaml
kubectl exec -it training-pod -- env | Select-String -Pattern
"batch_size|learning_rate|max_steps|model_arch|WANDB|HF_TOKEN"
```

Expected output

```
batch_size=128
learning_rate=3e-4
max_steps=50000
model_arch=llama3-8b
HF_TOKEN=demo-hf-token-not-real
WANDB_API_KEY=demo-wandb-key-not-real
```

Part 1b — Headless Services & StatefulSet DNS

Folder: 03_networking_config

Files: 06_headless_svc.yaml, 07_statefulset.yaml

What this teaches

A headless Service (clusterIP: None) allocates no virtual IP -- DNS for the Service name returns pod IPs directly, and each pod of the owning StatefulSet additionally gets its own stable per-pod hostname (worker-0.training-worker, worker-1.training-worker, ...). This is the backbone of distributed training: NCCL rendezvous needs every worker to dial every other worker by a stable hostname. A regular ClusterIP Service would make every worker resolve to the same virtual IP, and NCCL would hang at initialization.

How to run it

```
kubectl apply -f 06_headless_svc.yaml
kubectl apply -f 07_statefulset.yaml
kubectl exec worker-0 -- nslookup worker-1.training-worker.default.svc.cluster.local
kubectl exec worker-0 -- nslookup training-worker.default.svc.cluster.local
```

Expected output

```
Name:   worker-1.training-worker.default.svc.cluster.local
Address: 10.244.0.9                <- ONE specific pod

Name:   training-worker.default.svc.cluster.local
Address: 10.244.0.10
Address: 10.244.0.8
Address: 10.244.0.9                <- ALL pod IPs, no VIP
```

create a regular ClusterIP Service against the same pods, then delete it.

```
kubectl create service clusterip training-worker-demo-vip --tcp=29500:29500
kubectl patch service training-worker-demo-vip -p '{"spec":{"selector":{"app":"training-worker"}}}'
kubectl exec worker-0 -- nslookup training-worker-demo-vip.default.svc.cluster.local
kubectl delete service training-worker-demo-vip
```

```
Name:   training-worker-demo-vip.default.svc.cluster.local
Address: 10.96.168.200            <- ONE virtual IP, always
```

Part 1c — LoadBalancer for Production Endpoints

Folder: 03_networking_config

File: 08_loadbalancer_svc.yaml

What this teaches

ClusterIP is internal-only, NodePort is dev/debug-only (static port 30000-32767 on every node), and LoadBalancer is the production pattern: it provisions a real external IP via a cloud load balancer (ALB/NLB on AWS, equivalent elsewhere). This demo reuses the tiny-inference Deployment from 01_inference_service and exposes it externally.

How to run it

```
kubectl apply -f ../01_inference_service/tiny-inference-deployment.yaml
kubectl apply -f 08_loadbalancer_svc.yaml
kubectl get svc tiny-inference-lb
curl http://localhost:8080/health
```

Expected output

NAME	TYPE	EXTERNAL-IP	PORT(S)
tiny-inference-lb	LoadBalancer	172.18.0.5	8080:32023/TCP

```
{"status":"ok","device":"cpu"}
```

Discussion questions

- On a real cloud cluster, what actually happens in AWS/Azure/GCP when you create a LoadBalancer Service -- what API calls does the cloud controller manager make on your behalf?
- Why is NodePort described as dev/debug only -- what would break if you exposed a production inference endpoint this way?

Part 1d — Gateway API: the modern Ingress

Folder: 03_networking_config

Files: gatewayclass.yaml, gateway.yaml, httproute.yaml

What this teaches

Gateway API replaces Ingress as the standard HTTP/gRPC routing layer. It decouples the platform team, who own the GatewayClass and Gateway, from app teams, who own the HTTPRoute -- so an ML team can self-serve routing in their own namespace without cluster-admin access. Previously every team needed cluster-admin to touch Ingress; now the platform team creates one Gateway, and app teams attach HTTPRoutes to it.

One-time setup: Gateway API CRDs + Istio

Istio does not bundle the Gateway API CRDs, and it is also the controller that implements them here (Istio registers itself under controllerName: istio.io/gateway-controller). Both need installing once per cluster:

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.2.0/standard-install.yaml
istioctl install --set profile=demo -y
kubectl create namespace team-alpha
kubectl label namespace team-alpha istio-injection=enabled
```

How to run it

```
cd 03_networking_config
kubectl apply -f gatewayclass.yaml
kubectl apply -f gateway.yaml
kubectl apply -f httproute.yaml
kubectl get gatewayclass inference-gateway -o wide
```

Expected output

NAME	CONTROLLER	ACCEPTED	AGE
inference-gateway	istio.io/gateway-controller	True	3s

host port 80 is often already taken: On Docker Desktop's local Kubernetes, every LoadBalancer Service is published on the host by a small per-Service proxy container. Docker Desktop's own engine backend already listens on host port 80 for itself, so a second Service also asking for port 80 fails ("SyncLoadBalancerFailed ... exit status 125") and stays <pending> forever. Fix: check the port is actually free first (Get-NetTCPConnection -LocalPort 80 -State Listen in PowerShell) and move the Gateway's listener to an unused port if not -- this lab uses 8082. Also remove the demo profile's default istio-ingressgateway Deployment/Service in istio-system if it's squatting the same port; Gateway API makes it redundant anyway.

EXTERNAL-IP can be unreliable for a second LoadBalancer: Docker Desktop's local LB publishing reliably covers only the first LoadBalancer Service created on the cluster. A second one (like this Gateway) can report Programmed: True with an IP, yet nothing is actually listening. Don't trust the reported IP as gospel -- verify with `kubectl port-forward` instead:

```
kubectl port-forward -n team-alpha svc/inference-gateway-inference-gateway 18082:8082
curl http://localhost:18082/v1/completions
```

```
llm-service stable response
```

Part 2 — Istio Canary Rollout

Folder: 07_istio

Files: llm_service_base.yaml, 09_istio_destinationrule.yaml, 10_virtualservice.yaml, 11_inference_v2.yaml, patch.json (revert to 90/10), patch-promote.json (promote to 0/100)

What this teaches

Istio's service mesh gives four things for inference traffic with zero application code changes: automatic mutual TLS between all pods (zero-trust inside the cluster), weighted traffic splitting for canary rollouts, auto-injected observability (metrics/traces/access logs), and Envoy-level rate limiting. This demo focuses on the canary pattern: a DestinationRule defines named subsets (stable/canary) from a pod label, and a VirtualService splits traffic across those subsets by weight.

One-time setup: Prometheus + Kiali

From the istioctl release download's samples/addons directory (Istio itself must already be installed, per Part 1d above):

```
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.30/samples/addons/prometheus.yaml

kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.30/samples/addons/kiali.yaml
```

How to run it

Deploy the stable baseline, the routing rules, then the canary version:

```
cd 07_istio
kubectl apply -f llm_service_base.yaml           # stable (v1), 2 replicas
kubectl apply -f 09_istio_destinationrule.yaml
kubectl apply -f 10_virtualservice.yaml          # 90/10 split
kubectl apply -f 11_inference_v2.yaml           # canary (v2), 1 replica
```

Generate real traffic from inside the mesh and count the split (the sort/uniq counting runs inside the pod's own shell, so this line works unchanged in PowerShell, WSL, or Git Bash):

```
kubectl exec curl-client -n team-alpha -- sh -c 'for i in $(seq 1 40); do curl -s http://llm-
svc.team-alpha.svc.cluster.local:8080/; echo; done | sort | uniq -c'
```

Expected output

```
34 response from STABLE (v1)
 6 response from CANARY (v2)
```

Promote the canary to 100% of traffic:

```
kubectl patch virtualservice llm-inference-vs -n team-alpha --type=json --patch-file=patch-
promote.json
```

Re-run the same 40-request loop after promoting:

```
40 response from CANARY (v2)
```

Confirm mTLS is on by default:

```
istioctl x describe pod llm-svc-stable-8ffd75877-bzxzj -n team-alpha
```

```
Workload mTLS mode: PERMISSIVE
```

```
kubectl port-forward -n istio-system svc/kiali 20001:20001
# open http://localhost:20001/kiali/console/graph/namespaces?namespaces=team-alpha
```

Appendix A — Quick Command Reference

Demo	Run	Verify
1a. ConfigMap/Secret	<code>kubectl apply -f 04_configmap.yaml && bash create_secret.sh</code>	<code>kubectl exec training-pod -- env</code>
1b. Headless Svc + StatefulSet	<code>kubectl apply -f 06_headless_svc.yaml -f 07_statefulset.yaml</code>	<code>kubectl exec worker-0 -- nslookup ...</code>
1c. LoadBalancer	<code>kubectl apply -f 08_loadbalancer_svc.yaml</code>	<code>curl http://localhost:8080/health</code>
1d. Gateway API	<code>kubectl apply -f gatewayclass.yaml -f gateway.yaml -f httproute.yaml</code>	<code>kubectl get gatewayclass inference-gateway -o wide</code>
2. Istio canary	<code>kubectl apply -f 09_istio_destinationrule.yaml -f 10_virtualservice.yaml -f 11_inference_v2.yaml</code>	<code>curl loop + uniq -c</code>

Appendix B — Cleanup

```
kubectl delete pod training-pod --ignore-not-found
kubectl delete configmap train-config --ignore-not-found
kubectl delete secret ml-tokens --ignore-not-found
kubectl delete statefulset worker --ignore-not-found
kubectl delete svc training-worker tiny-inference-lb --ignore-not-found

kubectl delete namespace team-alpha --ignore-not-found
kubectl delete gatewayclass inference-gateway --ignore-not-found
kubectl delete -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.2.0/standard-install.yaml --ignore-not-found
istioctl uninstall --purge -y
kubectl delete namespace istio-system --ignore-not-found
```